



React – Kompleksowy przewodnik

Od podstaw do zaawansowanych koncepcji



19 tematów



Best practices

Przewodnik dla programistów React

Czym jest React?



Biblioteka

JavaScript UI Library

- ✓ Tworzenie interfejsów użytkownika
- ✓ Komponentowy podejście
- ✓ Deklaracyjna natura
- ✓ Otwarte źródło (Facebook)



Virtual DOM

Wirtualny model DOM

- ✓ Lekka reprezentacja DOM
- ✓ Reconciliation algorytm
- ✓ Efficient updates
- ✓ Minimalizacja DOM operacji



Single Page App

Aplikacje jednostronicowe

- ✓ Płynne przejścia
- ✓ Dynamiczny routing
- ✓ Routing klienta (React Router)
- ✓ Rich interaktywność

Tworzenie projektu

Tworzenie projektu z Vite

```
# Utwórz nowy projekt  
npm create vite@latest my-react-app
```

```
# Wybierz framework  
✓ React  
✓ JavaScript
```

```
# Zainstaluj i uruchom  
cd my-react-app  
npm install  
npm run dev
```

Dlaczego Vite?

- ⚡ Szybki start dev server
- ☐ Hot Module Replacement
- ☐ Optymalizacja produkcji

Struktura folderów

```
node_modules/  
  
public/  
  index.html  
  
src/  
  App.jsx  
  App.css  
  main.jsx  
  
.eslintrc.cjs  
  
package.json  
  
vite.config.js
```

 Kod umieszczaj w folderze **src/**

JSX - podstawy

Co to jest JSX?

Rozszerzenie składni JavaScript, które umożliwia pisanie kodu HTML w plikach JavaScript.

```
const element = <h1>Witaj React!</h1>;
```

Różnice JSX vs HTML

- `class` → `className`
- `onclick` → `onClick`
- `style="color: red"` → `style={{ color: 'red' }}`
- Samozamykające tagi wymagają /
- Jednolity element nadrzędny

Wyrażenia JavaScript

Zmienne i funkcje:

```
const name = "Jan";  
const greeting = `Witaj, {name}`;
```

Obliczenia:

```
const x = 10;  
const y = 20;  
<p>Suma: {x + y}</p>
```

Funkcje:

```
function formatDate(date) {  
  return `${date.toLocaleDateString()}`;  
}  
<div>{formatDate(new Date())}</div>
```

Komponenty funkcyjne

Components • Functions •
Reusability



Tworzenie komponentu

```
import React from 'react';

function Welcome({ name }) {
  return (
    <div className="greeting">
      <h1>Witaj, {name}!</h1>
    </div>
  );
}
```



Eksport i import

Eksport domyślny:

```
export default function Welcome() {
  return <h1>Witaj!</h1>;
}
```

Import:

```
import Welcome from './Welcome';
```



Kompozycja komponentów

Nestowanie komponentów:

```
function Card({ title, children }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      {children}
    </div>
  );
}
```

Wykorzystanie:

```
function App() {
  return (
    <Card title="Mój projekt">
      <p>Opis projektu</p>
      <Welcome name="Użytkownik" />
    </Card>
  );
}
```



Best practices

- Kompaktowe, reużywalne komponenty
- Jasne nazewnictwo (PascalCase)
- Single Responsibility Principle

Props - podstawy

Props • Data Passing •
Destructuring



Przekazywanie danych przez props

Definiowanie komponentu z props:

```
function UserCard(props) {  
  return (  
    <div className="card">  
      <h2>{props.name}</h2>  
      <p>Wiek: {props.age}</p>  
    </div>  
  );  
}
```

Wykorzystanie komponentu:

```
function App() {  
  return (  
    <UserCard name="Anna" age=28 />  
  );  
}
```



Ważne informacje

- Props są tylko do odczytu (read-only)
- Przekazywane z rodzica do dziecka
- Unidirectional data flow



Destrukturyzacja props

Wersja z destrukturyzacją:

```
function UserCard({ name, age }) {  
  return (  
    <div className="card">  
      <h2>{name}</h2>  
      <p>Wiek: {age}</p>  
    </div>  
  );  
}
```

Zawansowana destrukturyzacja:

```
function UserProfile({ user: { name, email }, isActive }) {  
  return (  
    <div>  
      <h3>{name}</h3>  
      <p>{email}</p>  
      {isActive && <span>Online</span>}  
    </div>  
  );  
}
```



Korzyści destrukturyzacji

- Bardziej czytelny kod
- Łatwiejsze do zrozumienia parametry
- Mniej powtarzalny kod

Children prop

Przykład użycia:

```
function Card({ children }) {
  return (
    <div className="card">
      {children}
    </div>
  );
}
```

Wykorzystanie:

```
<Card>
  <h2>Tytuł</h2>
  <p>Opis</p>
</Card>
```

Zalety children

- Elastyczne zagnieżdżanie
- Wygodne dla layoutów
- Umożliwia kompozycję

Default props

Sposób 1 -

W definicji:

```
function Button {
  text =
    "Kliknij"
} {
  return
    <button>
      {text}
    </button>;
}
```

Sposób 2 -

Component.defaultProps:

```
Button.defaultProps
= {
  text:
    "Kliknij"
};
```

Użycie:

```
<Button
/>
<Button
  text="Zapisz"
/>
```

Bezpieczeństwo

- Zabezpiecza przed undefined
- Łatwe rozszerzanie
- Zwiększa elastyczność

Prop drilling

Problem:

```
<App>
  <Parent
    data="...">
    <Child
      data-
        {data}>
      <Grandchild
        data-
          {data} />
    </Child>
  </Parent>
</App>
```

Rozwiązanie

- Context

API:

```
const
DataContext
=
  React.createContext();

<DataContext.Provider
  value=
    {data}>
  {/*
    komponenty
  */}
</DataContext.Provider>
```

Ostrzeżenie

- Prop drilling = trudności w utrzymaniu
- Używaj Context API lub Redux
- Zwiększa złożoność aplikacji

Renderowanie warunkowe

» Operator &&

Podstawowe użycie:

```
function
UserGreeting({
  isLoggedIn }) {
  return (
    <div>
      <h1>Witaj!</h1>
      {isLoggedIn &&
        <p>Zalogowany</p>}
    </div>
  );
}
```

Filtrowanie list:

```
{items.filter(item =>
  item.active).map(item
=> (
  <div key={item.id}>
    {item.name}</div>
  ))}
```

↔ Operator ternary

Podstawowe użycie:

```
function Greeting({
  isLoggedIn }) {
  return (
    <div>
      {isLoggedIn ? (
        <p>Witaj ponownie!
      </p>
      ) : (
        <p>Proszę się
        zalogować</p>
      )}
    </div>
  );
}
```

Zmiana klas:

```
<div className=
{isLoggedIn ?
  "primary" :
  "secondary"}>
  Treść
</div>
```

» Early return

Wczesny powrót:

```
function
UserProfile({ user
}) {
  if (!user) {
    return
    <p>Ładowanie...</p>;
  }

  return (
    <div>
      <h1>{user.name}</h1>
    </div>
  );
}
```

Walidacja danych:

```
function Product({
  product }) {
  if (product.price ≤
    0) {
    return null;
  }

  return <span>
    {product.price}
  </span>;
}
```

Conditional Rendering • Logical AND • Ternary • Early Return

Renderowanie list

Metoda map()

Podstawowe użycie:

```
const numbers = [1, 2, 3];

function NumberList() {
  return (
    <ul>
      {numbers.map((num, index) => (
        <li key={index}>{num}</li>
      ))}
    </ul>
  );
}
```

Z tablicy obiektów:

```
{items.map(item => (
  <div key={item.id}>
    <h3>{item.title}</h3>
    <p>{item.desc}</p>
  </div>
))}
```

Key prop

Prawidłowy key:

```
{items.map(item => (
  <div key={item.id}>
    {item.name}
  </div>
))}
```

Unikaj indeksów:

```
{items.map((item, index) => (
  <div key={index}>X</div>
))}
```

Filtrowanie list

Podstawowe filtrowanie:

```
const activeItems = items.filter(item =>
  item.isActive
);

{activeItems.map(item => (
  <div key={item.id}>{item.name}</div>
))}
```

Warunkowe renderowanie:

```
{items.filter(item =>
  item.price > 100
).map(item => (
  <div key={item.id}>
    {item.name}
  </div>
))}
```

Wskazówki

- Używaj map() do tworzenia elementów
- Każdy element musi mieć unikalny key
- Preferuj ID z danych nad index

Dlaczego key jest ważny?

- Pomaga React zidentyfikować elementy
- Optymalizuje aktualizacje DOM
- Umożliwia animacje i przejścia

Best practices

- Łącz filter() z map() w łańcuchu
- Filtrowaj przed renderowaniem
- Używaj memo() dla list statycznych

useState - podstawy



Deklaracja stanu

Podstawowa składnia:

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
    {count}
    </button>
  );
}
```

Dekonstrukcja tablicy:

```
const [stan, setStan] = useState(wartość początkowa);
// ↑ stan, ↑ setter
```



Setter i inicjalizacja

Aktualizacja stanu:

```
setCount(newValue);
setCount(current => current + 1);
```

Inicjalizacja z wartością:

```
useState(0);
useState("Tekst");
useState(true);
useState([]);
useState({});
```

Lazy initialization:

```
useState(() => {
  return expensiveCalculation();
});
```



Ważne zasady

- Stan jest asynchroniczny
- Setter wyzwała ponowny render
- Używaj lazy init dla obliczeń

useState - niemutowalność



Aktualizacja obiektów

✗ NIEZALECANE - mutacja:

```
user.name = "Nowy";  
setUser(user);
```

✓ ZALECANE - spread operator:

```
setUser({  
  ...user,  
  name: "Nowy",  
  age: user.age + 1  
});
```

Zagnieżdżone obiekty:

```
setUser({  
  ...user,  
  address: {  
    ...user.address,  
    city: "Warszawa"  
  }  
});
```

🚫 Dlaczego nie mutować?

- React nie wykrywa zmian
- Problemy z renderowaniem
- Trudne debugowanie



Aktualizacja tablic

Dodawanie elementu:

```
setItems([...items, newItem]);  
setItems([...items.slice(0, 1), newItem, ...items.slice(1)]);
```

Usuwanie elementu:

```
setItems(items.filter(item => item.id !== id));
```

Aktualizacja elementu:

```
setItems(items.map(item =>  
  item.id === id ? { ...item, name: "Nowy" } : item  
));
```



Best practices

- Zawsze twórz nowe obiekty
- Używaj spread operator (...)
- Używaj funkcji update dla prev state

Obsługa zdarzeń

onClick

Podstawowe użycie:

```
<button onClick={handleClick}>
  Kliknij
</button>
```

Z argumentem:

```
<button onClick={() => handleClick(1)}>
  Kliknij 1
</button>
```

Z event object:

```
<button onClick={(e) => handleClick(e, 1)}>
  Kliknij z event
</button>
```

Wskazówki

- Używaj camelCase dla nazw
- Funkcja jako referencja lub arrow
- e.preventDefault() w razie potrzeby

onChange

Input text:

```
<input
  type="text"
  value={value}
  onChange={(e) => setValue(e.target.value)}
/>
```

Checkbox:

```
<input
  type="checkbox"
  checked={checked}
  onChange={(e) =>
    setChecked(e.target.checked)}
/>
```

Select:

```
<select value={selected} onChange=
  {handleChange}>
  <option value="a">A</option>
  <option value="b">B</option>
</select>
```

Event Object

Właściwości event:

```
function handleClick(e) {
  e.preventDefault();
  e.stopPropagation();
  console.log(e.target);
  console.log(e.target.value);
  console.log(e.target.type);
}
```

Event types:

```
onClick
onSubmit
onFocus
onBlur
onKeyDown
```

Uwaga

- Wartości są w e.target
- Use e.preventDefault() dla formularzy

Formularze - kontrolowane komponenty



Input Text

Podstawowy przykład:

```
const [name, setName] = useState("");

<input
  type="text"
  value={name}
  onChange={(e) => setName(e.target.value)}
/>
```

Z walidacją:

```
<input
  value={email}
  onChange={(e) => setEmail(e.target.value)}
  className={errors.email ? "error" : ""}
/>
```



Checkbox i Radio

Checkbox:

```
<input
  type="checkbox"
  checked={checked}
  onChange={(e) => setChecked(e.target.checked)}
/>
<label>Akceptuję warunki</label>
```

Radio buttons:

```
<input
  type="radio"
  value="male"
  checked={gender === "male"}
  onChange={() => setGender("male")}
/>
```



Kluczowe zasady

- Zawsze ustawiaj value/checked z stanu
- Używaj onChange do aktualizacji stanu
- Stan jest źródłem prawdy (single source of truth)

Formularze - validacja

! Stan błędów

Struktura stanu błędów:

```
const [errors, setErrors] = useState({});

const validate = () => {
  const newErrors = {};
  if (!email) newErrors.email = "Email wymagany";
  if (!password) newErrors.password = "Hasło wymagane";
  setErrors(newErrors);
  return Object.keys(newErrors).length === 0;
};
```

Wyświetlanie błędów:

```
<input />
{errors.email && <span className="error">{errors.email}</span>}
```

✓ Obsługa submit

Podstawowy submit handler:

```
const handleSubmit = (e) => {
  e.preventDefault();
  if (validate()) {
    console.log("Formularz poprawny");
    console.log({ email, password });
    // Wyślij dane do API
  }
};
```

Formularz z submit:

```
<form onSubmit={handleSubmit}>
  <input name="email" />
  <input name="password" type="password" />
  <button type="submit">Zaloguj</button>
</form>
```

★ Best practices

- Validacja przy onChange lub onBlur
- Używaj e.preventDefault() dla formularzy
- Przechowuj błędy w osobnym stanie
- Wyświetl komunikaty użytkownikowi

useEffect - podstawy



Co to są side effects?

Przykłady side effects:

- Pobieranie danych z API
- Zapisywanie do localStorage
- Subskrypcje
- Ustawianie document.title
- Animacje

Składnia useEffect:

```
useEffect(() => {  
  // Side effect code  
}, []);
```



Dependency Array

Pusty array [] - raz przy montowaniu:

```
useEffect(() => {  
  fetchData();  
}, []);
```

Z wartością - przy zmianie zmiennej:

```
useEffect(() => {  
  updateTitle(count);  
}, [count]);
```

Bez array - przy każdym renderze:

```
useEffect(() => {  
  console.log("Render");  
});
```

useEffect - cleanup



Return Function

Podstawowa składnia cleanup:

```
useEffect(() => {  
  // Setup code  
  return () => {  
    // Cleanup code  
  };  
}, []);
```

Subskrypcja - unsubscribe:

```
useEffect(() => {  
  const subscription = dataSource.subscribe(data => {  
    setData(data);  
  });  
  return () => {  
    subscription.unsubscribe();  
  };  
}, []);
```

Zapobieganie memory leaks:

```
useEffect(() => {  
  const timer = setTimeout(() => {  
    setShowMessage(true);  
  }, 3000);  
  return () => clearTimeout(timer);  
}, []);
```



AbortController

Anulowanie fetch requests:

```
useEffect(() => {  
  const controller = new AbortController();  
  const fetchData = async () => {  
    try {  
      const response = await fetch(url, {  
        signal: controller.signal  
      });  
      setData(await response.json());  
    } catch (error) {  
      if (error.name !== 'AbortError') {  
        console.error(error);  
      }  
    }  
  };  
  fetchData();  
  return () => controller.abort();  
}, []);
```

Ważne zasady:

- Cleanup zawsze przy odmontowaniu
- Anuluj nieukończone requests
- Unsubscribe z subskrypcji
- Usuń event listeners

Pobieranie danych z API



Fetch z async/await

123

Podstawowy przykład:

```
useEffect(() => {
  const fetchData = async () => {
    setLoading(true);
    try {
      const response = await fetch('https://api.example.com/data');
      const data = await response.json();
      setData(data);
    } catch (error) {
      setError(error.message);
    } finally {
      setLoading(false);
    }
  };
  fetchData();
}, []);
```



Loading i Error States

State management:

```
const [data, setData] = useState(null);
const [loading, setLoading] = useState(false);
const [error, setError] = useState(null);
```

Renderowanie warunkowe:

```
{loading && <div>Ładowanie...</div>}
{error && <div className="error">{error}</div>}
{data && <div>{data.name}</div>}
```

AbortController integration:

```
useEffect(() => {
  const controller = new AbortController();
  // ... fetch with controller.signal
  return () => controller.abort();
}, []);
```

localStorage w React

Persystencja danych

Zapisywanie danych:

```
useEffect(() => {  
  localStorage.setItem('user', JSON.stringify(user));  
}, [user]);
```

Odczytywanie danych:

```
const [user, setUser] = useState(() => {  
  const saved = localStorage.getItem('user');  
  return saved ? JSON.parse(saved) : null;  
});
```

Usuwanie danych:

```
const clearData = () => {  
  localStorage.removeItem('user');  
  setUser(null);  
};
```

Lazy Initial State

Dlaczego lazy initialization?

- Unika błędu hydration
- Zapobiega problemom z SSR
- Optymalizuje wydajność
- Wykonywane tylko raz

Hook dla localStorage:

```
const useLocalStorage = (key, initialValue) => {  
  const [value, setValue] = useState(() => {  
    try {  
      const item = window.localStorage.getItem(key);  
      return item ? JSON.parse(item) : initialValue;  
    } catch (error) {  
      return initialValue;  
    }  
  });  
  // ... useEffect dla setValue  
  return [value, setValue];  
};
```

Best practices

- Używaj JSON.stringify() i JSON.parse()
- Obsługuj błędy parse
- Używaj lazy initialization
- Stwórz własny hook useLocalStorage

Lifting state up



Wspólny stan rodzica

Rodzic przechowuje stan:

```
function ParentComponent() {
  const [sharedState, setSharedState] = useState("");
  const handleChange = (value) => {
    setSharedState(value);
  };
  return (
    <div>
      <ChildA value={sharedState} onChange={handleChange} />
      <ChildB value={sharedState} onChange={handleChange} />
    </div>
  );
}
```

Dziecko odbiera dane i callback:

```
function ChildA({ value, onChange }) {
  return (
    <input value={value} onChange={(e) => onChange(e.target.value)} />
  );
}
```



Komunikacja komponentów

Przekazywanie danych w górę:

```
// Child → Parent
const Child = ({ onAdd }) => {
  const handleClick = () => {
    onAdd("nowy element");
  };
  return <button onClick={handleClick}>Add</button>;
};
```

Rodzic aktualizuje stan:

```
function Parent() {
  const [items, setItems] = useState([]);
  const handleAdd = (item) => {
    setItems([...items, item]);
  };
  return <Child onAdd={handleAdd} />;
}
```

Przykład: Formularz i lista

```
const App = () => {
  const [tasks, setTasks] = useState([]);
  const addTask = (task) => {
    setTasks([...tasks, task]);
  };
  return (
    <div>
      <TaskForm onSubmit={addTask} />
      <TaskList tasks={tasks} />
    </div>
  );
};
```

Podsumowanie i best practices



Kluczowe koncepcje

- ✓ React to biblioteka UI dla SPA
- ✓ Virtual DOM dla wydajności
- ✓ Komponenty funkcyjne z hooks
- ✓ Props dla przepływu danych
- ✓ State w useState, efekty w useEffect
- ✓ Niemutowalne aktualizacje stanu



Best practices

- ★ Używaj funkcji update dla prev state
- ★ Lazy initialization dla useState
- ★ Cleanup w useEffect return
- ★ AbortController dla fetch
- ★ Lifting state up zamiast prop drilling
- ★ Lazy initial state dla localStorage



Kolejne kroki

- ➔ Context API dla globalnego stanu
- ➔ React Router dla nawigacji
- ➔ useReducer dla złożonego stanu
- ➔ useMemo i useCallback
- ➔ Custom hooks dla reużywalności
- ➔ Zarządzanie bibliotekami zewnętrznymi